

LECTURE 5

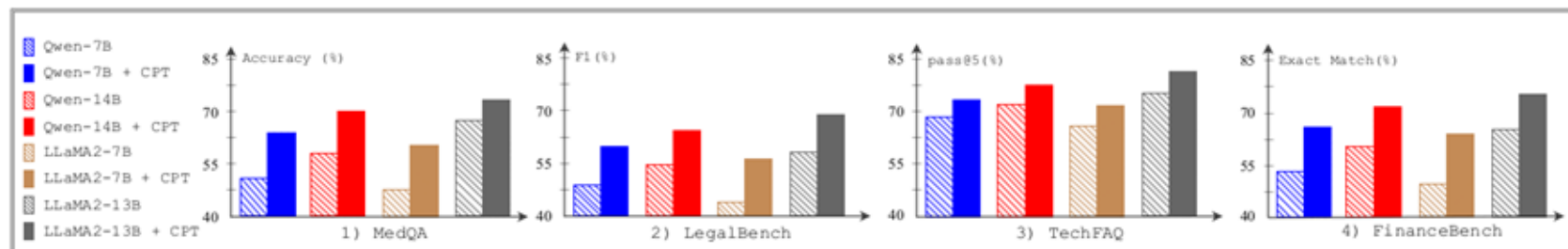
Post-training I: supervised fine-tuning

Gleb Molodtsov
5 Oct, 2026



After pretraining: knowledge or target behavior

The base model is already trained on clean, high-quality data.



Intervention

Add domain exposure through CPT

Domains

Medical, legal, technical, financial

Then

Repeat SFT and evaluate

Now choose: domain knowledge through continued pre-training, or target behavior through SFT.



Post-training signals

01

CPT

More text
Domain knowledge

02

SFT

Target outputs
Desired behavior

03

Preferences

Answer comparisons
Relative quality

04

RL

Sampled outcomes
Successful actions

Context methods

ICL and prompt search change the request, while these stages train parameters.



Lecture roadmap

01 SFT objective

02 Instruction tuning

03 Data is the SFT algorithm

04 Training and diagnosis

05 What SFT can forget

06 Where adaptation is stored

07 In-context learning

08 Prompt adaptation

09 Teacher forcing

10 Frontier training pipelines

11 Why move beyond demonstrations?



01

- **SFT objective**



A base LM continues text

CONTEXT $x_{<t}$

The cat sits on the



NEXT TOKEN x_t

mat

$P(\text{next token} \mid \text{context})$



SFT learns from input–target pairs

Classification

“I loved it”



positive

Structured output

Extract name and date



```
{"name": "Anna",  
"date": "May 12"}
```

Tool use

Find order #1842



lookup_order(1842)



Many examples teach instruction following

TRAIN

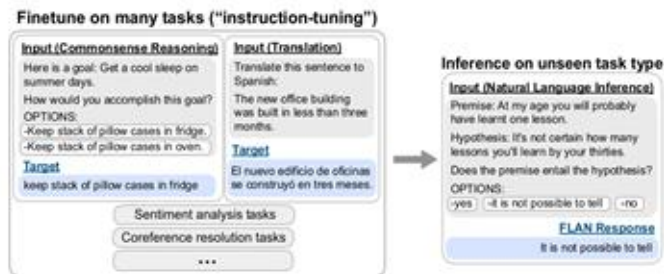
Mix many tasks, each expressed as an instruction with a target answer.

TRANSFER

On a new task type, the model infers the required action from the instruction alone.

Main takeaway:

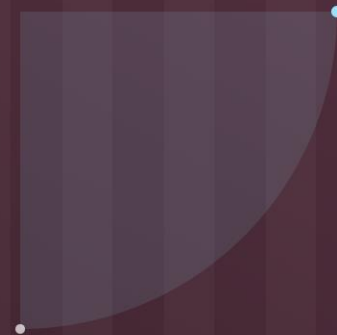
Instruction tuning teaches a transferable protocol: read the instruction → perform the task.



02

• Instruction tuning

Next-token objectives, masks, and chat templates



Same objective, different target data

Pretraining

Ordinary text



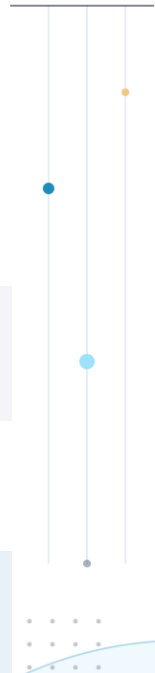
Predict its continuation

SFT

Input + chosen response



Predict selected target tokens



The loss mask selects the target tokens

Visible context

Supervised target

USER

$6 \times 7 = ?$

ASSISTANT

42

EOS

$$L = -(1/N) \sum_t m(t) \log p(x(t) \mid x(<t))$$



Different masks teach different behavior



	User	Analysis	Tool call	Tool result	Final
Full sequence	loss	loss	loss	loss	loss
Completion*	context	loss	loss	loss	loss
Assistant only	context	loss	loss	context	loss
Last turn	context	context	context	context	loss
Selective	context	context	loss	context	loss

*Completion begins after User in this example.

Select supervision by the task and verification quality. Tool observations usually stay context.



Chat templates define the generation boundary

Training sequence

USER: Explain LoRA
ASSISTANT: LoRA adds ... EOS

Inference prefix

USER: Explain LoRA
ASSISTANT:

Inspect one real batch after tokenization

User context

Assistant answer

EOS

PAD

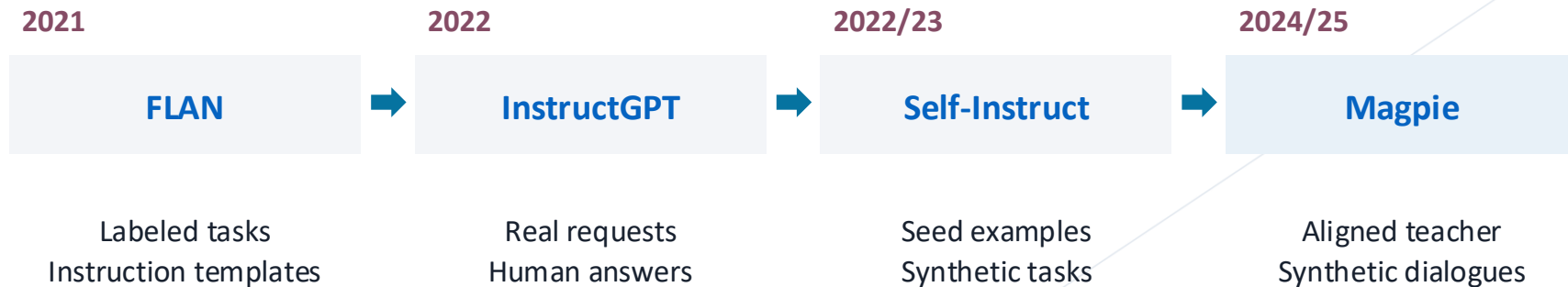


03

- Data is the SFT algorithm



Where SFT demonstrations come from



Each source provides **different coverage** and inherits different errors.



InstructGPT: demonstrations, then human preferences

1 · Demonstrations

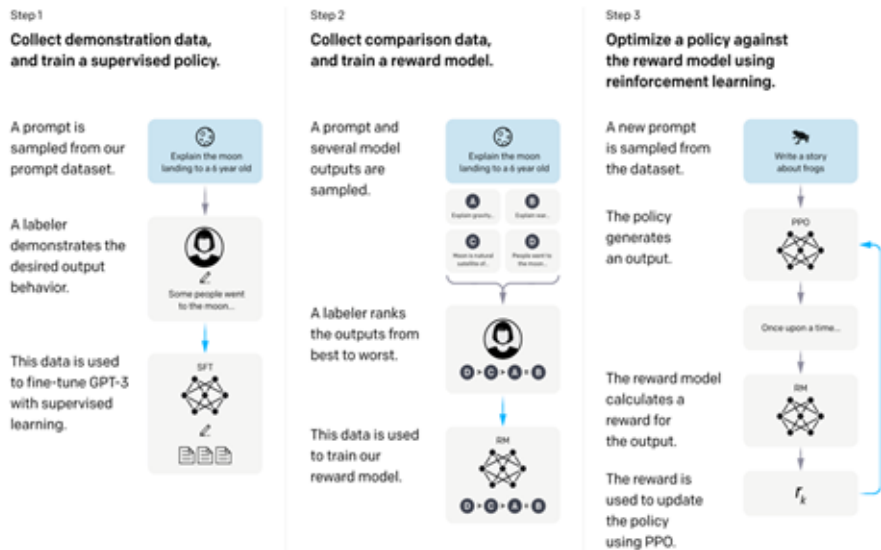
API requests → human answers → SFT policy.

2 · Preferences

Rank outputs → reward model → PPO policy updates.

3 · Result and scope

1.3B InstructGPT preferred to GPT-3 175B on API prompts.



Self-Instruct: bootstrap an instruction dataset

1 · Start small

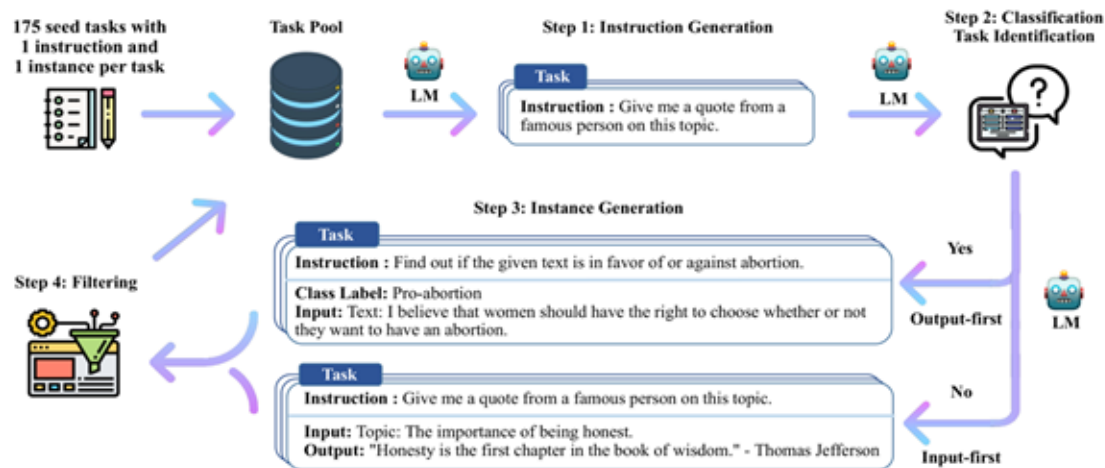
175 human seed tasks → new instructions and examples.

2 · Generate + filter

Classify tasks → generate instances
→ filter invalid / similar data.

3 · Scale, with noise

52k instructions / 82k instances;
54% fully valid in a 200-item audit.



Magpie: let an aligned model generate the request

1 · Open the user turn

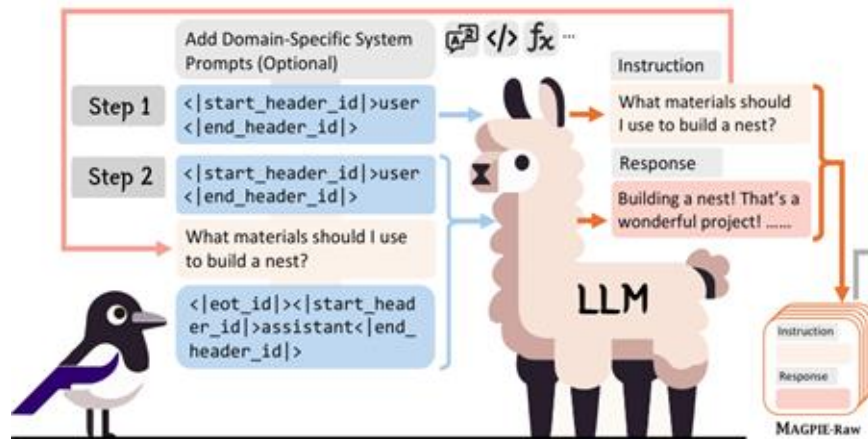
Pre-query template only → model samples a user request.

2 · Generate the answer

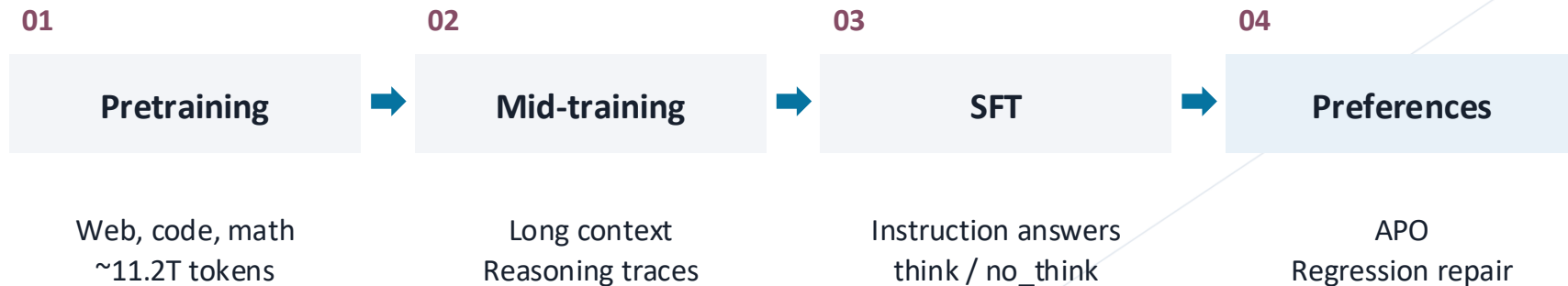
Request + chat template → model generates the response.

3 · Select for SFT

4M generated pairs; filter quality, difficulty and near-duplicates.



SmolLM3: the full training pipeline



Each stage can add skills and **change capabilities learned earlier**.



SmolLM3 changes its pretraining mixture

Early

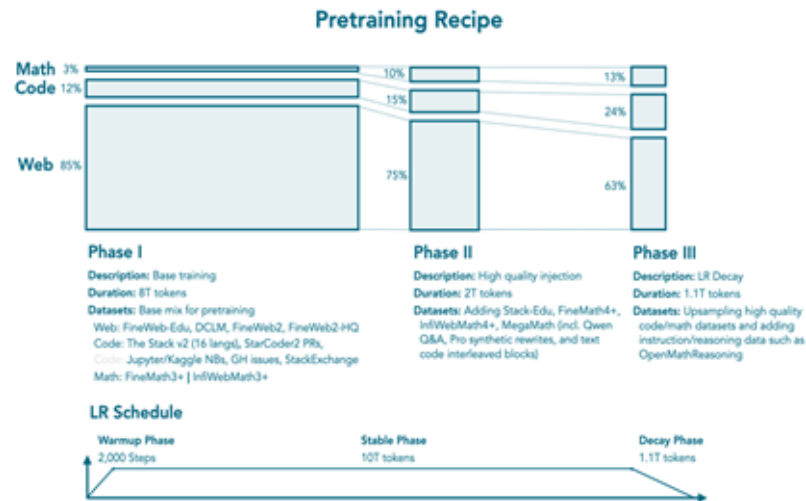
Web / code / math: 85 / 12 / 3

Middle

75 / 15 / 10

Late

63 / 24 / 13



Code and math gain weight over training. The schedule alone **does not establish causality**.



Long context and reasoning use separate stages

Long Context Training



Base: 4k

During pretraining a context length of 4k tokens was used. The long context training used the same data. **~8 pages of text**



Step 1: 32k

The RoPE theta was increased to 1.5M and training continued for 50B tokens with 32k context size. **~64 pages of text**



Step 2: 64k

The RoPE theta was further increased to 5M and training continued for 50B tokens with 64k context size. **~128 pages of text**



Context

100B tokens:
4k → 32k → 64k

Reasoning

35B unique tokens,
four epochs

Retention

Evaluate both skills
after later stages



SmolLM3 SFT teaches two response modes

Targets

A trace or a direct answer

Coverage

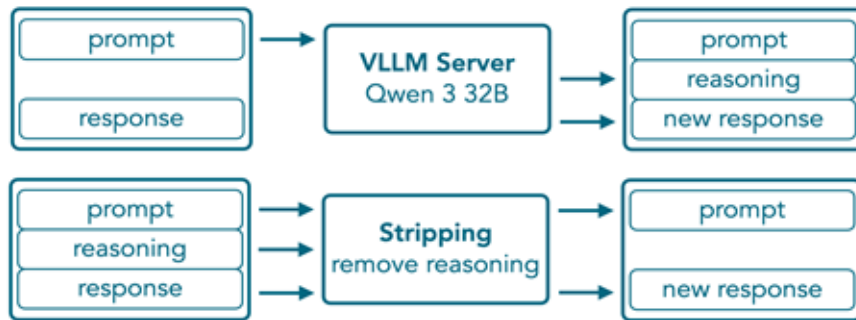
Math, code, languages, tools

Mixture

1.8B tokens across 22 datasets

Synthetic SFT Data

For dual reasoning mode to work reliably it was necessary to generate synthetic data for datasets which had not reasoning traces and create a reasoning stripped version of reasoning datasets:



Assistant responses receive loss. User messages and tool observations remain context.



One skill, two supervised answer formats

/no_think

Question: What is 6×7 ?



42

/think

Question: What is 6×7 ?



$6 \times (5 + 2) = 30 + 12 = 42$

Both modes cover broad tasks. Teachers add missing traces; trace removal creates direct answers.



Preference training requires regression checks



APO

Prefer chosen over rejected answers

Regression

Long-context RULER scores fell

Repair

90% APO + 10% long-context checkpoint



04

• Training and diagnosis

A recipe that reveals what caused the result



Choosing the starting checkpoint

Base model

New protocol or very different behavior



Build the assistant behavior

Instruct model

Behavior already matches the task



Adapt while checking retention



More pretraining can reduce performance after SFT

Controlled comparison

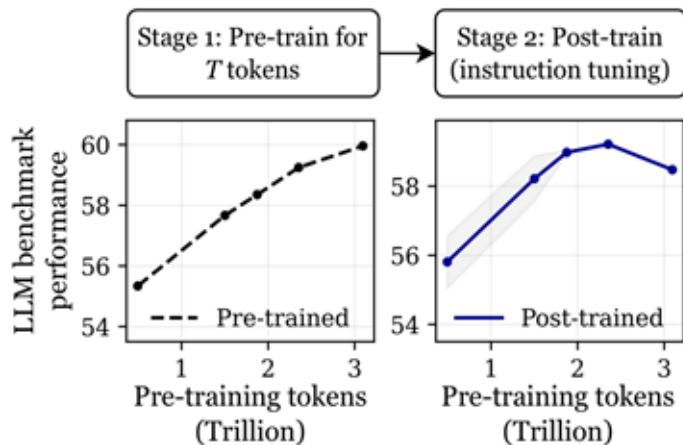
OLMo-1B checkpoints at different pretraining budgets.

Observed pattern

Base scores rise. Post-SFT scores peak, then fall.

Selection rule

Compare adapted checkpoints with their own LR search.



Choose the checkpoint by **post-SFT quality and retention**, rather than pretraining score alone.



What we tune in SFT

Setting

Learning rate

Effective batch

Training duration

Schedule + warmup

Optimizer + decay

Length + packing

Loss mask

Effect and decision

Update size and adaptation / retention balance

Gradient averaging and updates per data budget

Adaptation strength and checkpoint selection

LR trajectory, chosen together with duration

Update rule and regularization

Context, truncation and example boundaries

Which tokens define the supervised objective

Assistant-only loss

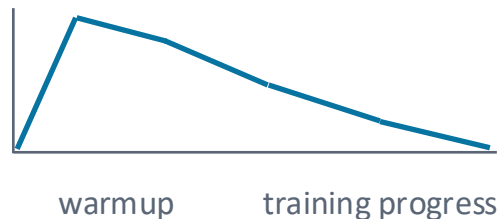
USER: What is 6×7 ?

ASSISTANT

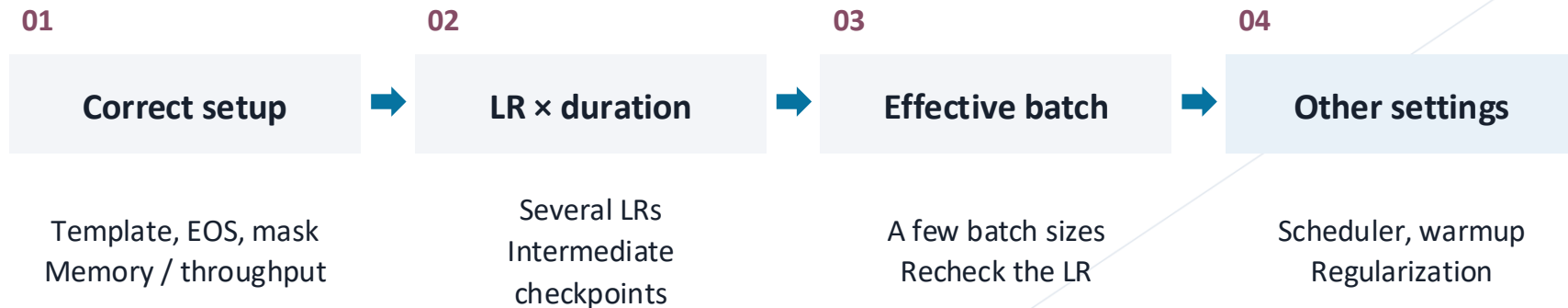
42 EOS

Gray: context Blue: loss tokens

Illustrative LR schedule



A practical order for tuning SFT



Effective batch = microbatch per GPU × accumulation steps × data-parallel GPUs

Lower LR can preserve more general capability

Question

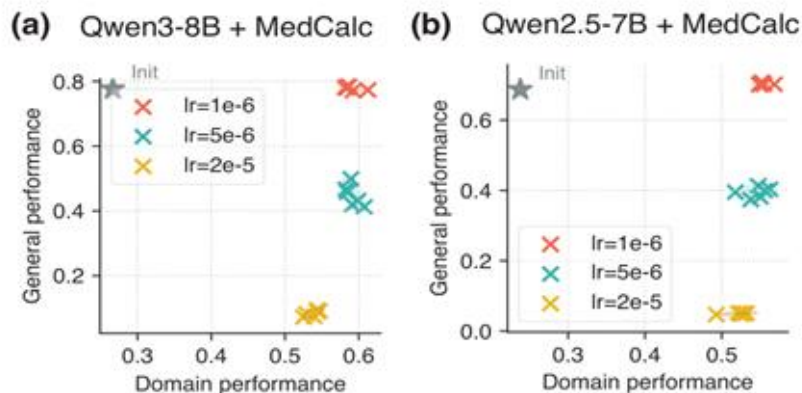
Can domain SFT improve the task while retaining general skills?

Experiment

Qwen3-8B / Qwen2.5-7B on MedCalc.
Best domain checkpoint per run.

Reading the plot

LR $1e-6$ gives a better trade-off than $5e-6$ or $2e-5$ here.



Small batches can work well with a tuned recipe

Controlled setup

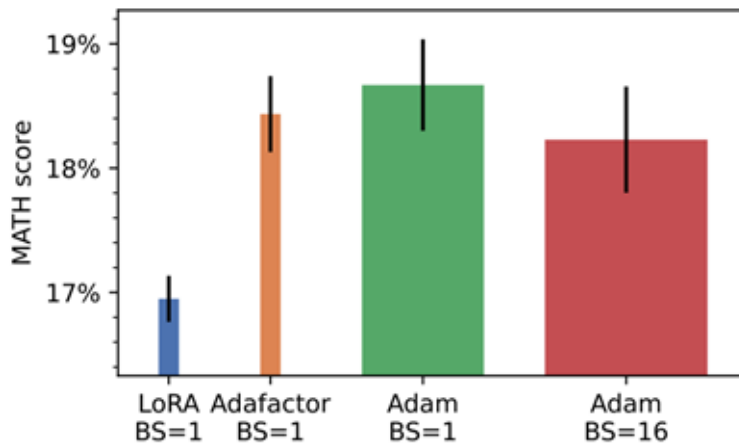
Gemma 3 4B on MATH. Five epochs and eight learning rates.

Result

Full SFT with batch 1 matches batch 16 in this experiment.

Important condition

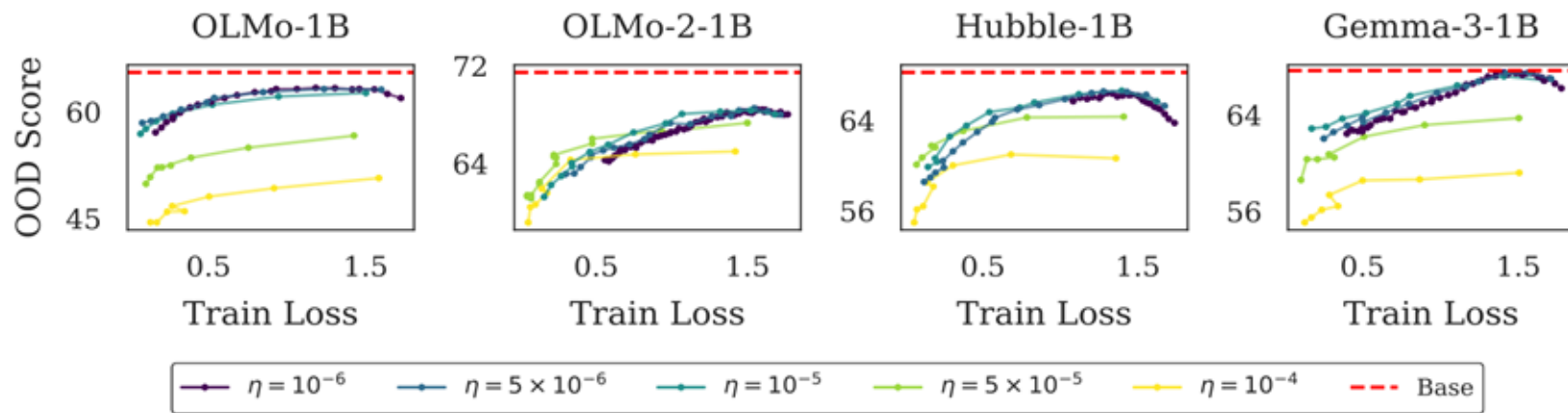
Adam / Adafactor second-moment decay was scaled with batch size.



Choose batch size for **quality and hardware efficiency**. Large gradient accumulation needs evidence.



Equal SFT loss can hide different amounts of forgetting



Hold fixed

Same SFT training loss
Different learning rates

Compare

Four 1B model families
Anthropic-HH instruction data

Evaluate separately

General-task performance
Alongside domain quality



05

- **What SFT can forget**

General skills, safety alignment, and ways to preserve them



Three different failure modes

What changes

Training overfit

Longer SFT fits its training set better

Catastrophic forgetting

Adaptation changes previous behavior

Catastrophic overtraining

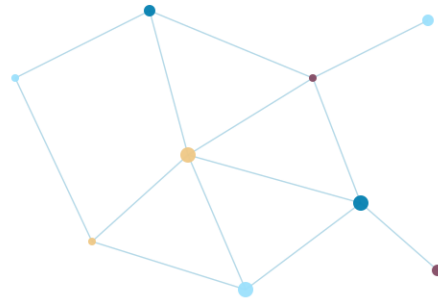
More pretraining changes adaptability

What reveals it

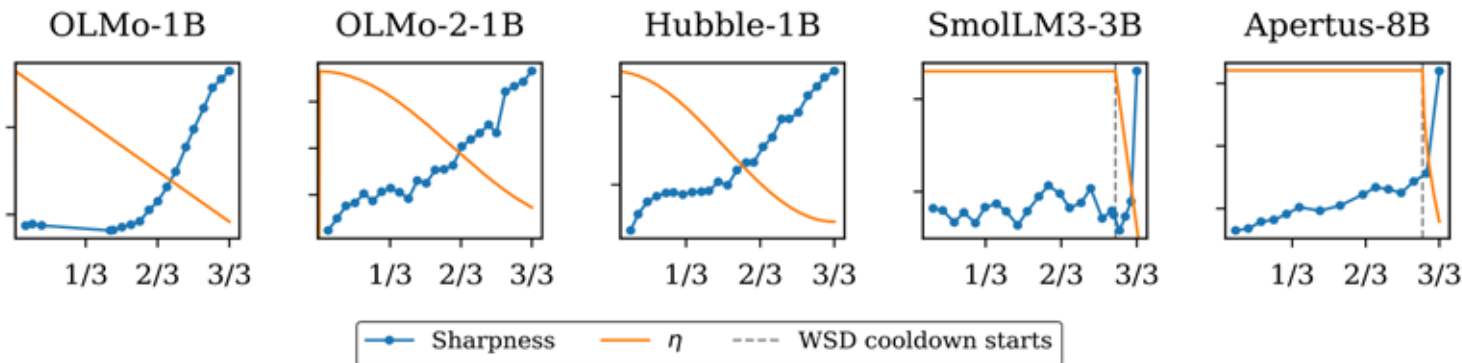
Target validation performance falls

Old tasks or safety degrade after SFT

Later base checkpoints fare worse after SFT



Pretraining cooldown can change later SFT sensitivity



Blue curve

Sharpness of the
pretraining loss landscape

Orange curve

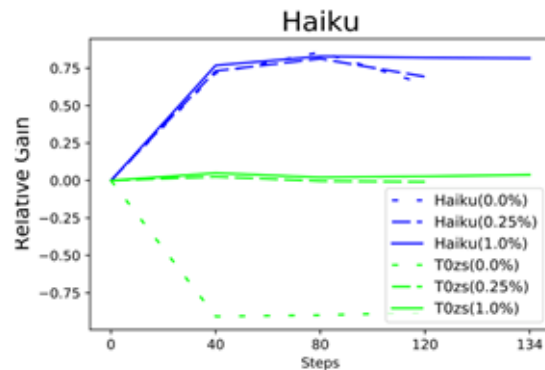
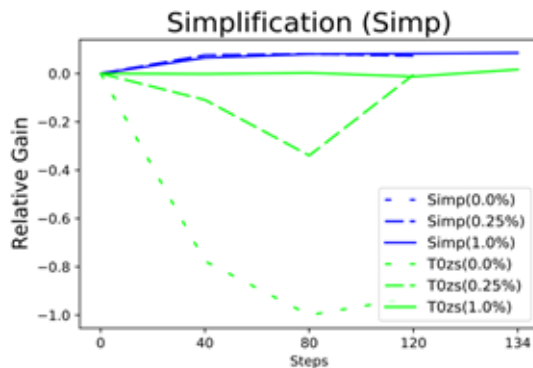
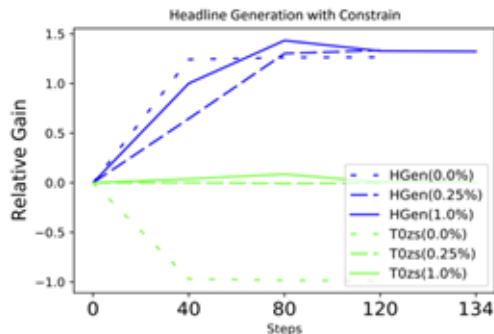
Pretraining learning rate
WSD cooldown marked

Proposed mechanism

Greater local sensitivity
More forgetting after adaptation



Rehearsal can preserve earlier skills during adaptation



Blue curves

New task performance
Similar across replay levels

Green curves

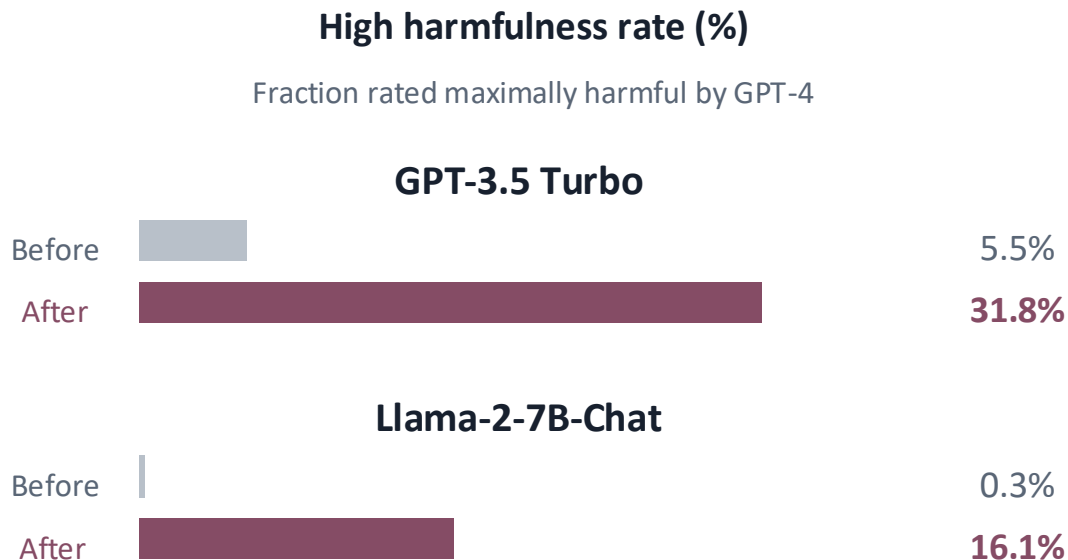
Previous T0 zero-shot tasks
Degrade without rehearsal

Controlled intervention

0%, 0.25% and 1%
Stored prior-task data



Benign SFT can weaken safety alignment



06

• PEFT

Weights, adapters, and context



LoRA trains a low-rank weight update

Frozen

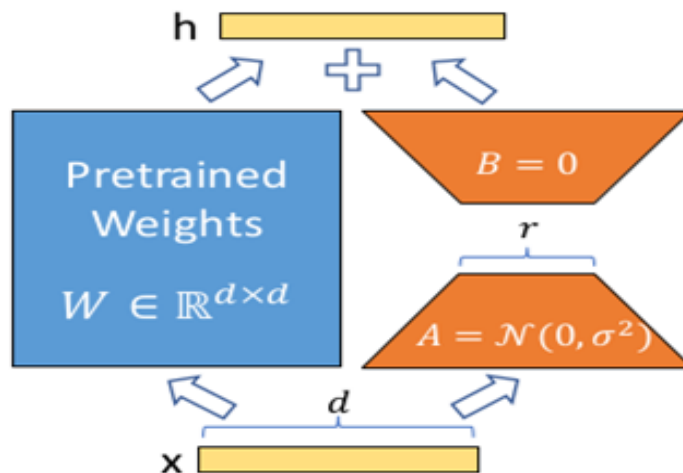
The original weight matrix W

Trainable

$\Delta W = (\alpha/r)$ BA

Example

4096^2 , rank 8: 65k vs 16.8M values



LoRA rank trades adaptation against retention

Higher rank

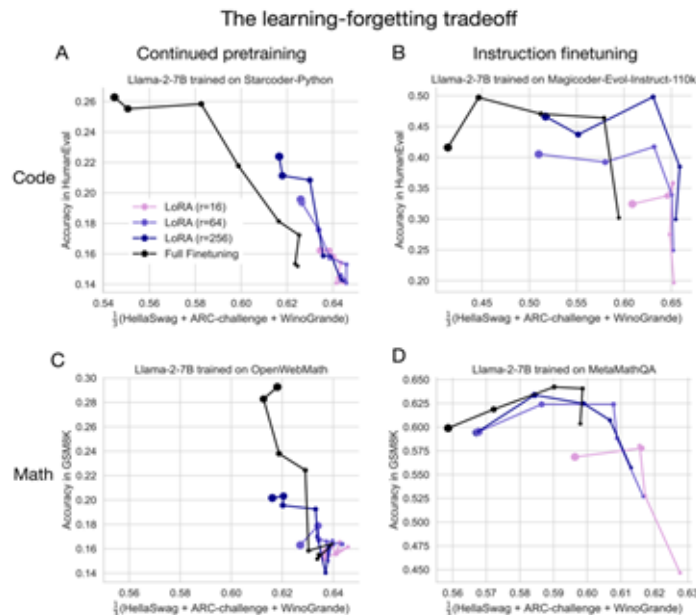
A more expressive update

Two scores

New task quality and old skill retention

Selection

Look for the upper-right region



LoRA variants change different parts of training

AdaLoRA

Rank allocation

$$r_1 \neq r_2 \neq r_3$$

Estimate importance; prune singular components to reallocate a fixed budget.

LoRA+

Optimization

$$\text{LR}(\text{A}) \neq \text{LR}(\text{B})$$

Same A/B factors; tune the LR ratio to improve feature learning.

DoRA

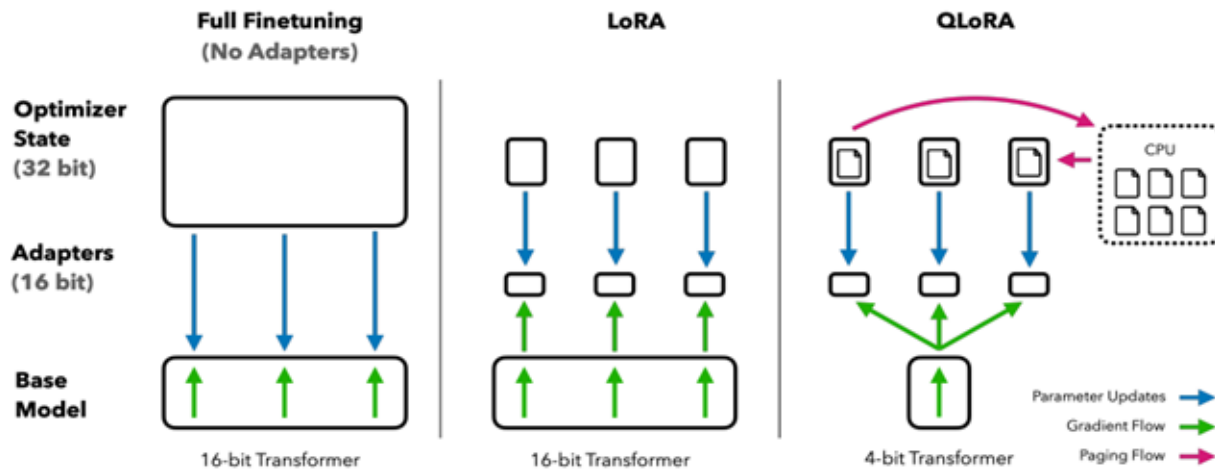
Parameterization

$$\mathbf{W} = \mathbf{m} \times \mathbf{V} / ||\mathbf{V}||$$

Train magnitude m separately; LoRA adapts the normalized direction $\mathbf{V}$.



QLoRA combines a 4-bit base with LoRA



07

- **In-context learning**



Behavior can also change through context

Weight adaptation

Train parameters



Behavior persists in the checkpoint

Context control

Add instructions or examples



Behavior is conditioned on this request



ICL puts demonstrations in the prompt

Zero-shot

Classify sentiment.

The movie was wonderful.

Sentiment:

Few-shot

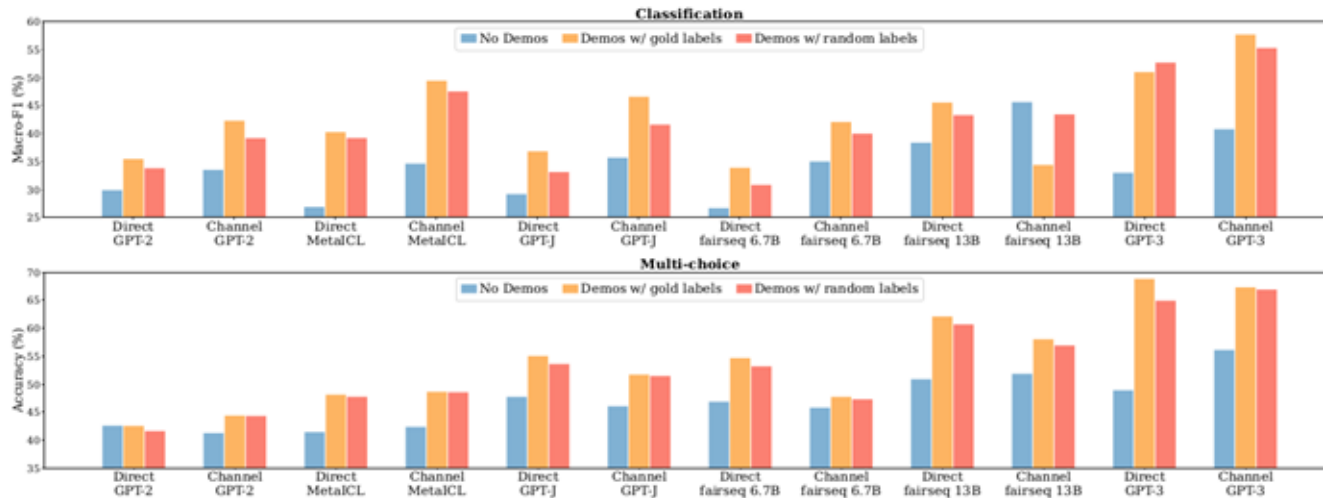
I hated it. → negative

Absolutely great. → positive

The movie was wonderful. →



Do ICL demonstrations need correct labels?



Examples reveal the task being continued

Prior structure

Biographies connect names and nationalities

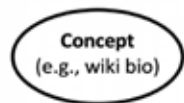
Evidence

Einstein → German; Gandhi → Indian

New query

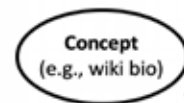
Marie Curie → Polish

1. Pretraining documents are conditioned on a **latent concept** (e.g., biographical text)



Albert Einstein was a German theoretical physicist, widely acknowledged to be one of the greatest physicists of all time. Einstein is best known for developing the theory of relativity, but he also

2. Create **independent examples** from a **shared concept**. If we focus on full names, wiki bios tend to relate them to nationalities.



Input (x)	Output (y)	Delimiter
Albert Einstein was	German	\n
Mahatma Gandhi was	Indian	\n
Marie Curie was	?	...brilliant? ...Polish?

3. Concatenate examples into a **prompt** and predict next word(s). **Language model (LM)** implicitly infers the **shared concept** across examples despite the unnatural concatenation

Albert Einstein was German \n Mahatma Gandhi was Indian \n Marie Curie was

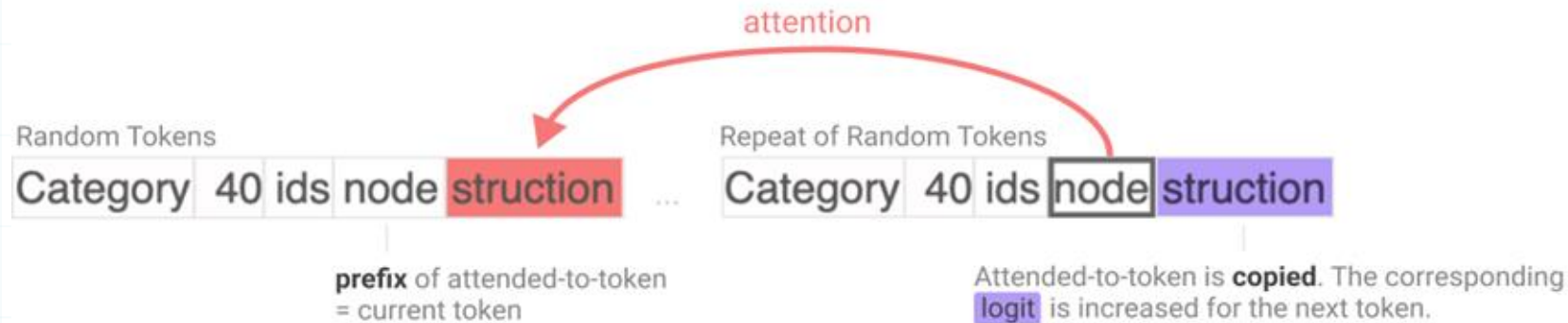


Polish

Bayesian view: $p(y \mid x, D) = \sum_z p(y \mid x, z) p(z \mid D)$. D : prompt examples; z : **latent task**.



Induction heads match a prefix and copy its continuation



Match

Find the earlier occurrence of the prefix

Retrieve

Attend to the token that followed it

Copy

Increase that continuation's probability

This circuit explains a **specific copying mechanism**, not every form of ICL.

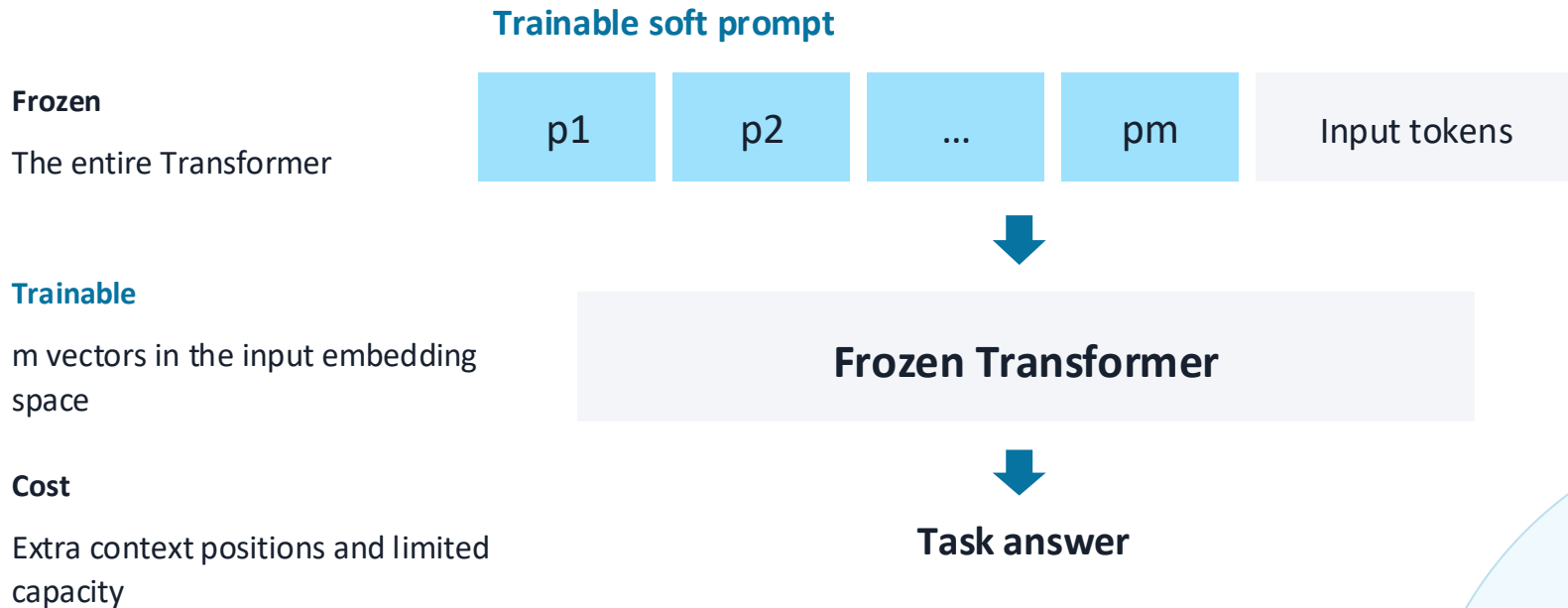


08

- **Prompt adaptation**



Prompt tuning learns soft input vectors



ICL uses token examples; prompt tuning finds **soft context** through optimization.



Prefix tuning adds learned attention memory

Frozen

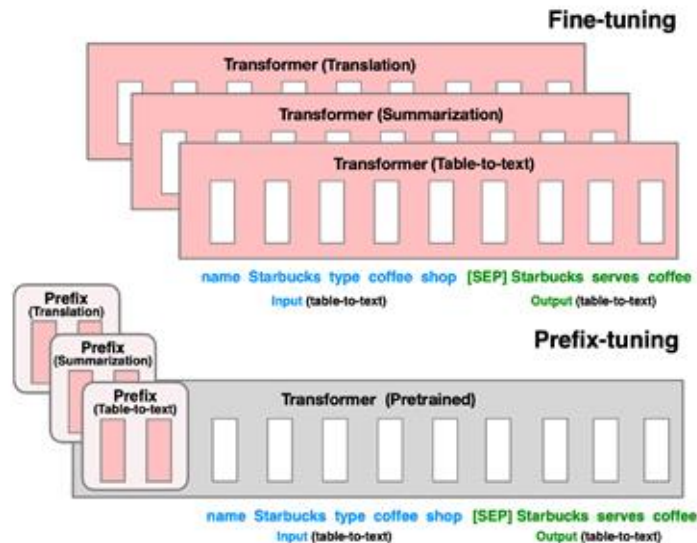
One shared Transformer

Trainable

Task-specific K/V prefixes at layers

Inference

Prefixes remain in the KV cache



GEPA uses rollout feedback to edit instructions

Run

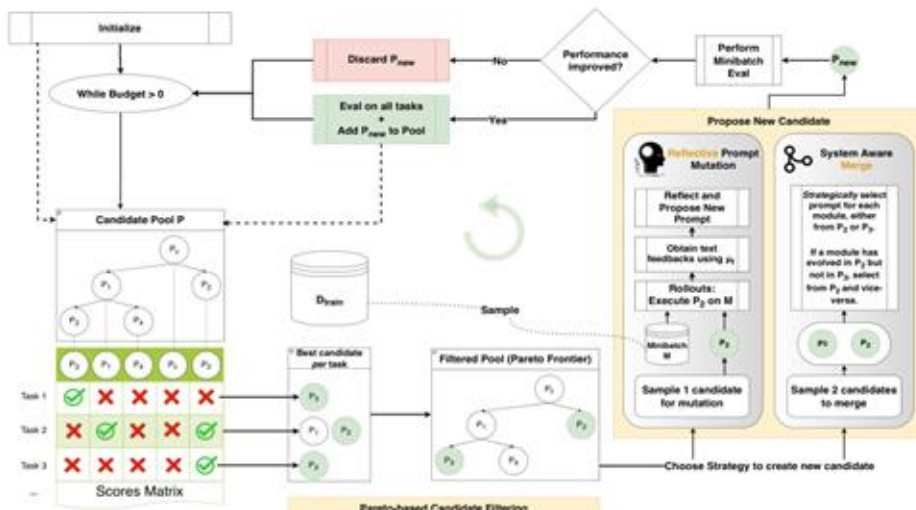
Collect a trace and evaluator feedback

Reflect

Locate a failure in the instructions

Search

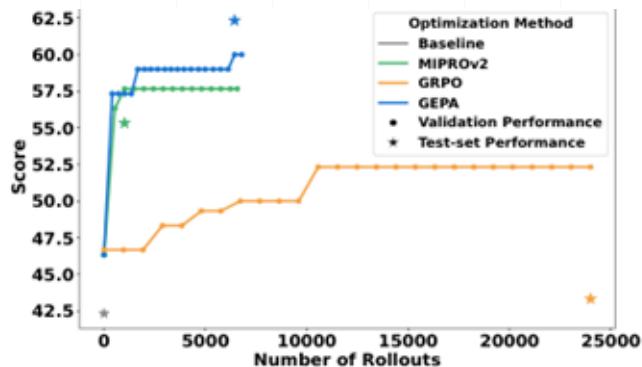
Edit, evaluate, and retain candidates



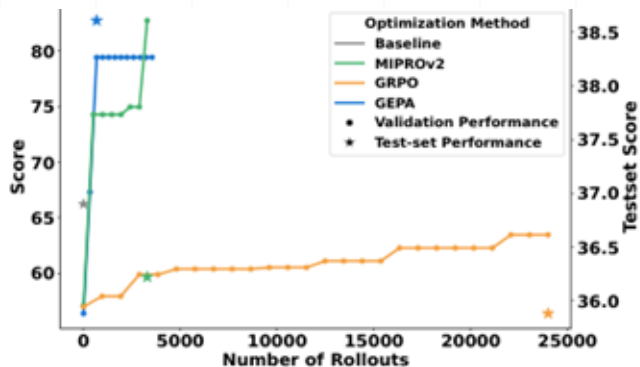
The search changes system text while keeping the base model **weights fixed**.



GEPA can improve with fewer rollouts



(a) HotpotQA, Qwen3 8B



(b) IFBench, Qwen3 8B



GEPA depends on useful textual feedback

Good conditions

Informative traces + reliable evaluator



Instructions explain the failure

Limitations

Missing base capability or weak feedback



Text search cannot supply every skill



09

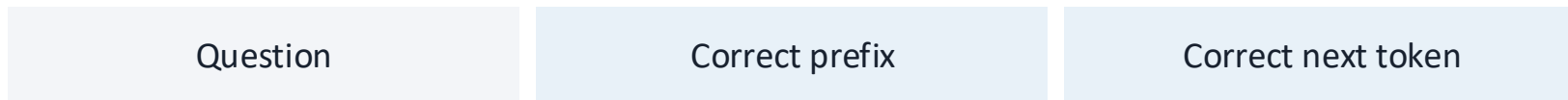
• Teacher forcing

What training loss misses about generation

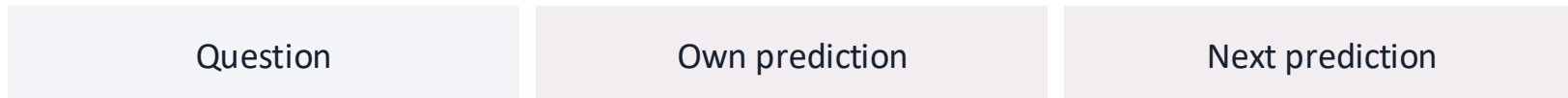


Training and generation use different histories

Training: gold previous tokens



Generation: the model's own previous tokens



Self-generated errors are harder to correct

EXTERNAL ERROR

USER message
Injected wrong step



Model response
Correction success: p_{ext}

INTERNAL ERROR

ASSISTANT history
Same injected wrong step

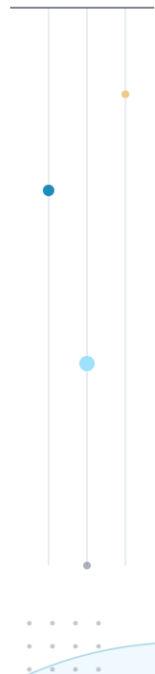


Model continuation
Correction success: p_{int}

MEAN RELATIVE GAP

64.5%

Offline SFT traces may mismatch test-time errors; on-policy RL trains on the model's own attempts.



10

- **Frontier training pipelines**

DeepSeek, Qwen, Tülu, Granite, and Rufus-Air



Reasoning SFT changes the demonstrated trajectory

Answer-only target

What is 6×7 ?



42

Trace target

What is 6×7 ?



$6 \times (5 + 2) = 30 + 12 = 42$



DeepSeek: two different engineering questions

Feasibility

R1-Zero

Base → reasoning RL
Can reasoning emerge?

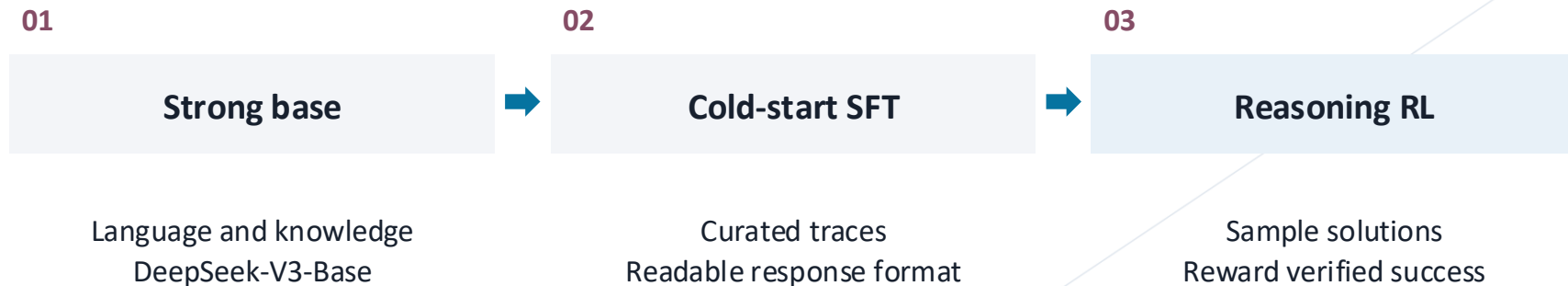
Assistant recipe

R1

SFT → RL → selected SFT → RL
Can it become a broad assistant?



Cold-start SFT initializes readable reasoning



Repeated SFT consolidates selected RL successes

Generate

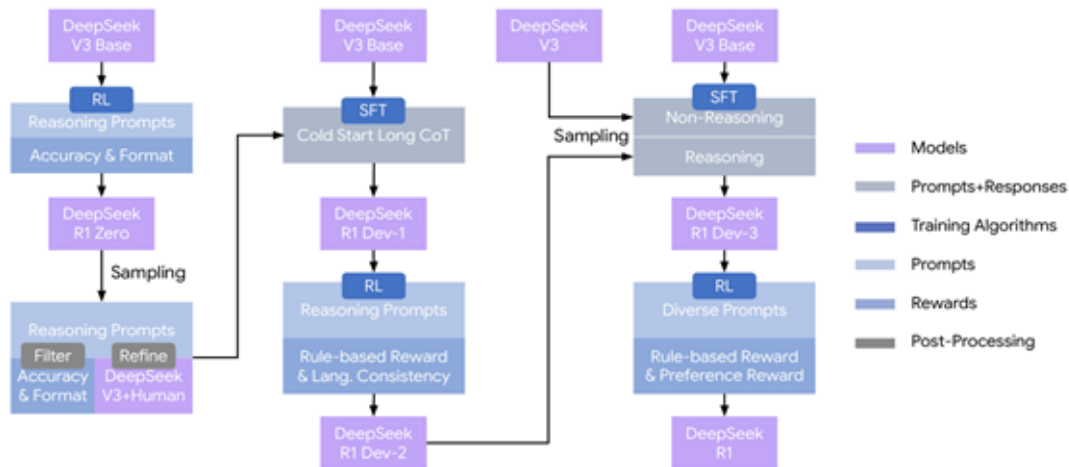
The RL policy samples new answers

Select

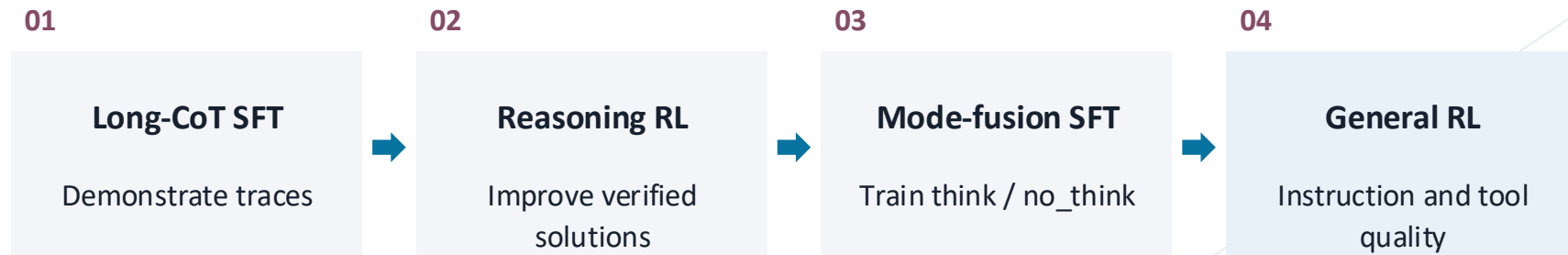
Filter successful, readable trajectories

Mix

Add reasoning and ordinary tasks



Qwen3 learns both reasoning and direct-answer modes



Mode-fusion SFT teaches two response formats

/think

Reasoning trace → final answer

/no_think

Direct answer without a visible trace



A larger thinking budget can improve accuracy

X-axis

Thinking-token budget

Y-axis

pass@1 on four tasks

Cost

More inference tokens and time

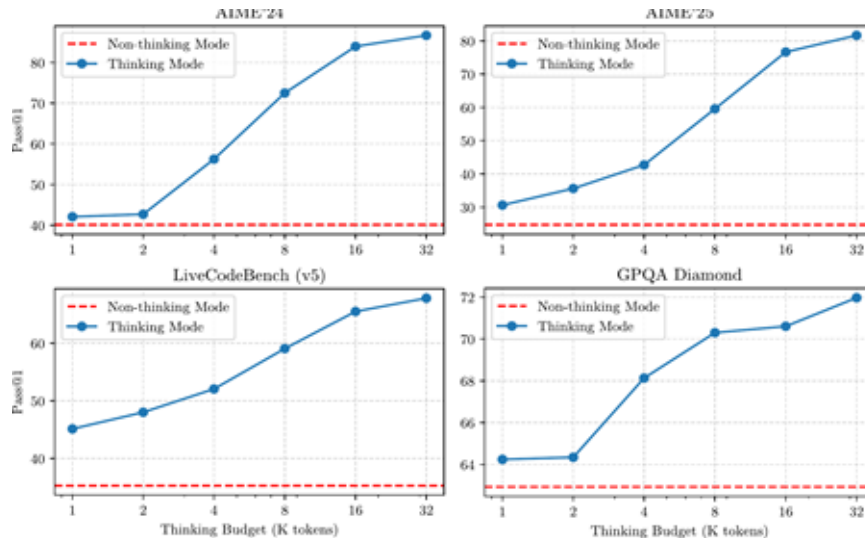


Figure 2: Performance of Qwen3-235B-A22B with respect to the thinking budget.

This is **test-time compute scaling**, not another weight-training stage.



Granite SFT includes tool trajectories

Granite 4.2 SFT mixture

68.4% general skills

31.6% agentic

User task

Tool call

Observation

Final answer

7.2M examples · ~100B corpus tokens · ~65B supervised tokens · 2 epochs



11

- **Why move beyond demonstrations?**

Comparisons and outcomes provide additional training signals



Demonstrations and preferences supervise different things

SFT: one demonstrated answer

Prompt x + target response y_+



Increase $\log \pi_{\theta}(y_+ | x)$

Preferences: compare two answers

Prompt x + chosen y_+ + rejected y_-

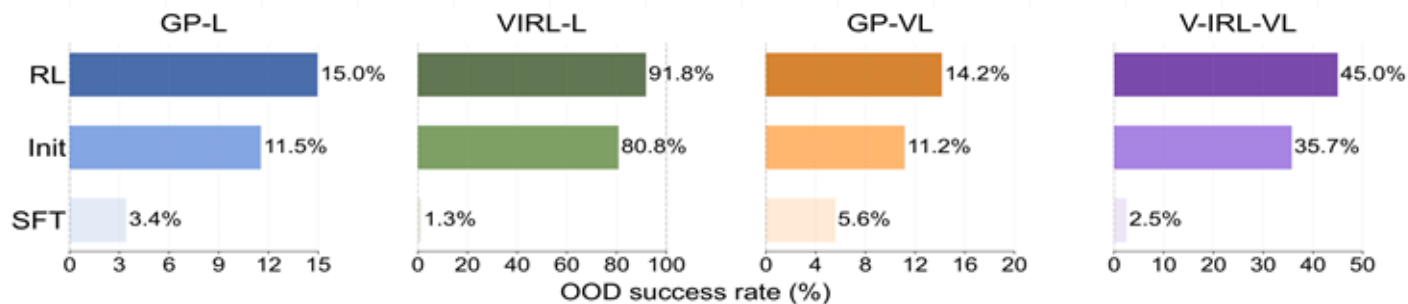


Learn which response is preferred

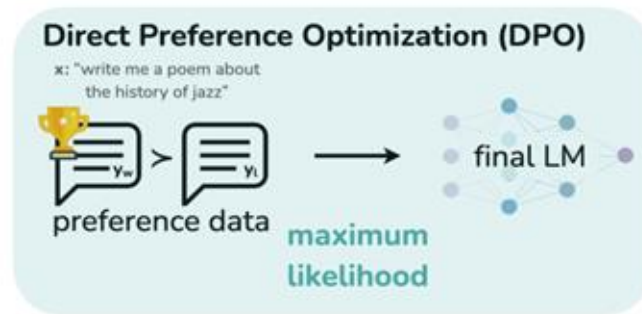
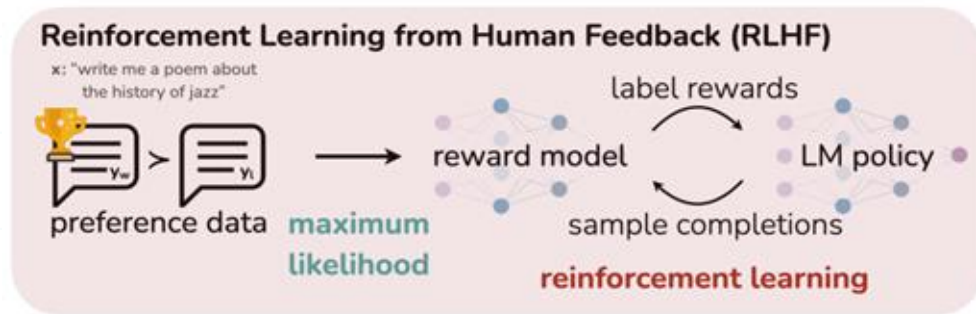
Chosen-only SFT uses the positive answer. Preference training also uses the **rejected alternative**.



Outcome feedback can improve transfer to new task variants



Preference optimization after an SFT initialization



SFT: five takeaways

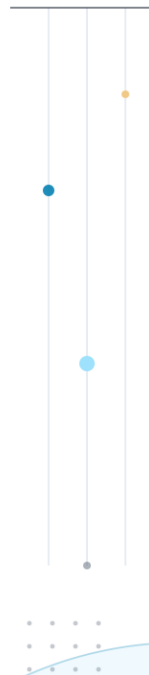
SFT learns from demonstrated responses

Data and checkpoint choice shape adaptation

Learning rate and duration belong in the same search

Context, LoRA, and full SFT offer different trade-offs

Preferences and outcomes add different training signals





NEXT LECTURE

RL for LLMs

Reward models, DPO, and controlling policy drift



Reading list · Foundations & instruction data

• **FLAN: Finetuned Language Models Are Zero-Shot Learners**

Wei et al., 2021 · How instruction tuning transfers to unseen tasks.

arXiv ↗

• **InstructGPT: Learning to follow instructions**

Ouyang et al., 2022 · Demonstrations, human preferences, and the SFT → RLHF pipeline.

arXiv ↗

• **Self-Instruct**

Wang et al., 2023 · Bootstrap instruction data from a small set of seed tasks.

ACL ↗

• **Magpie**

Xu et al., 2024 · Generate instruction–response pairs from an aligned model.

arXiv ↗

• **LIMA: Less Is More for Alignment**

Zhou et al., 2023 · What a small, carefully curated SFT dataset can achieve.

arXiv ↗

• **The False Promise of Imitating Proprietary LLMs**

Gudibande et al., 2023 · Distinguish imitation of style from transfer of capabilities.

arXiv ↗

• **s1: Simple test-time scaling**

2025 · Two studies of small reasoning datasets; compare data selection and evaluation.

s1 ↗

• **LIMO: Less Is More for Reasoning**

LIMO ↗



Reading list · Checkpoints & training settings

Overtrained Language Models Are Harder to Fine-Tune

Springer et al., 2025 · Starting checkpoint quality and downstream adaptability.

PMLR ↗

SFT Doesn't Always Hurt General Capabilities

Lin et al., 2025 · Learning rate and the adaptation–retention trade-off.

arXiv ↗

Small Batch Size Training for Language Models

Marek et al., 2025 · Small batches under an appropriately tuned training recipe.

arXiv ↗

(How) Learning Rates Regulate Catastrophic Overtraining

Rofin et al., 2026 · Equal training loss can conceal different capability retention.

arXiv ↗

Why Supervised Fine-Tuning Fails to Learn

Xue et al., 2026 · A systematic study of incomplete learning in LLMs.

ACL ↗



Reading list · Retention, feedback & SFT limitations

Fine-tuned Language Models are Continual Learners

Scialom et al., 2022 · Sequential adaptation and retention of previous skills.

[ACL ↗](#)

Fine-tuning Aligned Language Models Compromises Safety

Qi et al., 2023 · Alignment can degrade even under apparently benign fine-tuning.

[arXiv ↗](#)

Self-CorrectionBench

Tsui et al., 2025 · Test whether feedback actually improves a model's own answer.

[arXiv ↗](#)

SCoRe: Training Language Models to Self-Correct via RL

Kumar et al., 2024 · Multi-turn reinforcement learning for self-correction.

[ICLR ↗](#)

Direct Preference Optimization

Rafailov et al., 2023 · Learn from preferred vs. rejected answers without explicit RL.

[arXiv ↗](#)

SFT Memorizes, RL Generalizes

Chu et al., 2025 · Compare generalization in the paper's reasoning and visual tasks.

[PMLR ↗](#)



Reading list · Parameter-efficient fine-tuning

LoRA: Low-Rank Adaptation of Large Language Models

Hu et al., 2021 · Train low-rank weight updates instead of all model parameters.

arXiv ↗

QLoRA: Efficient Finetuning of Quantized LLMs

Dettmers et al., 2023 · Quantized base weights plus trainable low-rank adapters.

arXiv ↗

LoRA Learns Less and Forgets Less

Biderman et al., 2024 · Compare LoRA and full fine-tuning across adaptation and retention.

arXiv ↗

AdaLoRA

LoRA+

DoRA

AdaLoRA ↗

LoRA+ ↗

DoRA ↗

2023–2024 · Adaptive rank allocation, separate adapter learning rates, and weight decomposition.

The Power of Scale for Parameter-Efficient Prompt Tuning

Lester et al., 2021 · Learn soft prompts while keeping the backbone frozen.

arXiv ↗

Prefix-Tuning: Optimizing Continuous Prompts for Generation

Li & Liang, 2021 · Learn a continuous prefix rather than update the model weights.

ACL ↗

PEFT-Bench

Belanec et al., 2026 · A benchmark for comparing parameter-efficient fine-tuning methods.

ACL ↗



Reading list · Learning & optimization through context

- • • **Language Models are Few-Shot Learners**

- • • Brown et al., 2020 · Task adaptation through examples in the prompt.

[arXiv ↗](#)

- • • **Rethinking the Role of Demonstrations**

- • • Min et al., 2022 · Which parts of in-context examples drive performance?

[arXiv ↗](#)

- • • **An Explanation of In-context Learning as Implicit Bayesian Inference**

- • • Xie et al., 2021 · A theoretical account of inference from context.

[arXiv ↗](#)

- • • **In-context Learning and Induction Heads**

- • • Olsson et al., 2022 · Mechanistic evidence for circuits that copy contextual patterns.

[Article ↗](#)

- • • **GEPA: Reflective Prompt Evolution Can Outperform RL**

- • • Agrawal et al., 2025 · Optimize prompts from execution feedback and textual reflection.

[arXiv ↗](#)



Reading list · Modern pipelines & practical references



SmolLM3

Hugging Face, 2025 · An open recipe for data mixtures, SFT, and reasoning behavior.

Report ↗

DeepSeek-R1

DeepSeek-AI, 2025 · RL-first exploration, cold-start data, and filtered-rollout SFT.

arXiv ↗

Qwen3 Technical Report

Qwen Team, 2025 · Thinking/non-thinking behavior and a staged post-training pipeline.

arXiv ↗

Tülu 3: Pushing Frontiers in Open Language Model Post-Training

Lambert et al., 2024 · Open SFT, preference optimization, and verifiable-reward training.

arXiv ↗

Granite 4.2

IBM, 2026 · A contemporary model release and post-training design choices.

Report ↗

Rufus Air

2026 · A recent pipeline case study discussed in the lecture.

arXiv ↗

TRL: SFTTrainer documentation

Hugging Face · Chat templates, assistant-only loss, packing, and training configuration.

Docs ↗

